

UD5: Gestión de usuarios

Índice

Gestión de cuentas de usuarios.....	2
Creación de una cuenta de usuario.....	3
Creación de usuarios locales.....	3
Creación de usuarios comunes.....	4
Edición de una cuenta de usuario.....	6
Borrado de una cuenta de usuario.....	6
Concesión de privilegios.....	7
Retirada de privilegios.....	8
Consulta la información de usuarios.....	9
Gestión de roles y perfiles.....	11
Creación, asignación y revocación de roles.....	11
Default roles.....	11
Creación de roles comunes.....	11
Asignación de roles comunes.....	12
Sinónimos.....	12
PROXY USER.....	13

Gestión de cuentas de usuarios

Es la cuenta de usuario el mecanismo para controlar lo que una persona o aplicación puede hacer en la base de datos. Para controlar la actividad de los usuarios, a las cuentas de éstos se les conceden los diferentes privilegios para explotar la base de datos, ya sea con fines de gestión o de administración.

Cuando se conecta a una base de datos multitenant, la gestión de usuarios y privilegios es un poco diferente a los entornos tradicionales de Oracle. En entornos multitenant existen dos tipos de usuarios.

- **Usuario común:** el usuario está presente en todos los contenedores (raíz y todos los PDB).
- **Usuario local:** el usuario solo está presente en una PDB específica. El mismo nombre de usuario puede estar presente en varios PDB, pero no están relacionados.

Para facilitar la concesión de privilegios se usan los **roles**, cuya concesión conlleva la concesión de todos los privilegios que el rol posee.

Asimismo, existen dos tipos de roles.

- Rol común: está presente en todos los contenedores (raíz y todos los PDB).
- Rol local: el rol solo está presente en una PDB específica. Se puede utilizar el mismo nombre de función en varias PDB, pero no están relacionadas.

En cuanto al modo en el que una cuenta de usuario puede explotar el sistema, se usa lo que se denomina **perfil**. Un perfil tendrá una configuración del uso de la arquitectura del sistema, por ejemplo, porcentaje de uso del procesador, cantidad de memoria principal por utilizar, número máximo de procesos, horquilla de uso de la máquina, etcétera.

En la tecnología multitenant en la que existe un contenedor principal o raíz que contiene todos los objetos comunes a todas las bases de datos hijas. Esta base de datos se denomina global o base de datos contenedor (CDB, container database), ya que es la que contiene a todas las bases de datos. Las bases de datos hijas, llamadas bases de datos conectables (PDB, pluggable database), se comportan como unidad independiente, como si fuesen una base de datos única.

Con relación a los usuarios, roles y privilegios, estos se gestionan en el contenedor raíz o principal, o en un contenedor local. De este modo, este tipo de objetos se pueden gestionar como locales en un PDB o globales en un CDB.

Creación de una cuenta de usuario

Para crear una cuenta de usuario se usa un nombre. Este nombre de usuario no debe existir en el sistema y se le debe asignar una contraseña. Estos dos son los parámetros obligatorios que deben definirse, pero existen otros opcionales recomendables. La sintaxis para la creación de una cuenta de usuario es la siguiente:

```
CREATE USER nombre IDENTIFIED {BY contraseña | EXTERNALLY}
[DEFAULT TABLESPACE nombre_tablespace]
[TEMPORARY TABLESPACE nombre_espacio_temporal]
[QUOTA {valor [K | M] | UNLIMITED} ON nombre_tablespace...]
[PROFILE nombre_perfil]
[PASSWORD EXPIRE]
[ACCOUNT {LOCK | UNLOCK}]
[[CONTAINER=ALL | CURRENT]]
```

Creación de usuarios locales

- Debes estar conectado con un usuario con el privilegio CREATE USER
- El nombre de usuario del usuario local no debe tener el prefijo "C##" o "c##".
- El nombre de usuario debe ser único dentro del PDB.
- Puedes especificar la cláusula CONTAINER=CURRENT u omitirla, ya que esta es la configuración predeterminada cuando el contenedor actual es un PDB.

El siguiente ejemplo tiene que modificar la contraseña en la primera conexión.

```
CREATE USER manuel IDENTIFIED BY oracle DEFAULT TABLESPACE users
TEMPORARY TABLESPACE temp QUOTA UNLIMITED ON users PASSWORD EXPIRE;
```

PRÁCTICA

CREACIÓN DE USUARIOS

Crea los usuarios DirectorVentas, DirectorCompras y VendedorCaja01 en el contenedor XEPDB1. El tablespace Tienda debe estar creado. Si no es así, créalo tal como se indicó en el tema de almacenamiento. Recuerda que para trabajar en el contenedor XEPDB1 se debe ejecutar el comando ALTER SESSION SET CONTAINER=XEPDB1;.

A continuación, usando el comando CREATE USER, crea las cuentas de usuario siguientes:

- a) El usuario DirectorVentas tiene la contraseña satnev y su tablespace predeterminado es users, con cuota ilimitada.
- b) El usuario DirectorCompras tiene la contraseña sarpmoc, que debe ser modificada en el momento de la primera conexión. Su tablespace es TiendaVirtual y tiene una cuota de 300 megabytes.
- c) El usuario VendedorCaja01 tiene la contraseña ajac, que debe ser modificada en el momento de la primera conexión. Su tablespace por defecto es TiendaVirtual y tiene una cuota de 50 megabytes. La cuenta está bloqueada en estos momentos.

El nombre de un usuario que se usa para un PDB se puede usar en diferentes PDB, aunque en realidad, son usuarios diferentes, con identificadores distintos. La siguiente consulta, que debe ser ejecutada desde el contenedor raíz, muestra los usuarios creados en un entorno multicontenedor:

```
SELECT USER_ID , USERNAME FROM CDB_USERS;
```

Para conocer el nombre del contenedor activo se puede usar el comando **SHOW CON_NAME**. El nombre del contenedor raíz es CDB\$ROOT. En este contenedor no se puede crear un usuario local.

PRÁCTICA

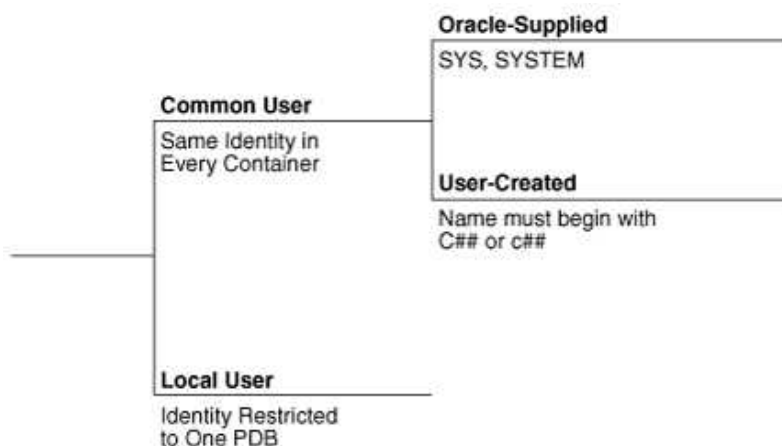
SELECCIÓN DE USUARIOS

- Mira la vista CDB_USERS y DBA_USERS, selecciona los usuarios, que diferencia hay?, mira si hay un campo que indique si es un usuario común o no. Seleccionalos de nuevo.
- El campo con_id se refiere al contenedor en el que está creado. Haz un join para seleccionar todos los usuarios junto con el nombre del contenedor en el que están creados.
- Cierra una pluggable database, vuelve a hacer el select. Están los usuarios correspondientes a esa pluggable? Que quiere decir eso en cuando a dónde están guardados los metadatos de los usuarios de la pluggable?.

Creación de usuarios comunes

Para crear un usuario común debemos:

1. Estar conectado como un usuario común que tenga el privilegio CREATE USER
2. El contenedor actual debe ser el contenedor raíz.
3. El nombre de usuario para el usuario común debe tener el prefijo "C##" o "c##" y contener solo caracteres ASCII o EBCDIC.
4. El nombre de usuario debe ser único en todos los contenedores.
5. Los DEFAULT TABLESPACE, TEMPORARY TABLESPACE, QUOTA y PROFILE deben hacer referencia a objetos que existan en todos los contenedores
6. Puedes especificar la cláusula CONTAINER=ALL u omitirla, ya que esta es la configuración predeterminada cuando el contenedor actual es el raíz.



Así pues, cuando se crea un usuario en el contenedor raíz, este se crea en todas las bases de datos, tanto en la raíz como en todas las PDB. Los usuarios globales o comunes deben comenzar con el texto C##, excepto SYS y SYSTEM.

El siguiente ejemplo crea un usuario común en todos los contenedores:

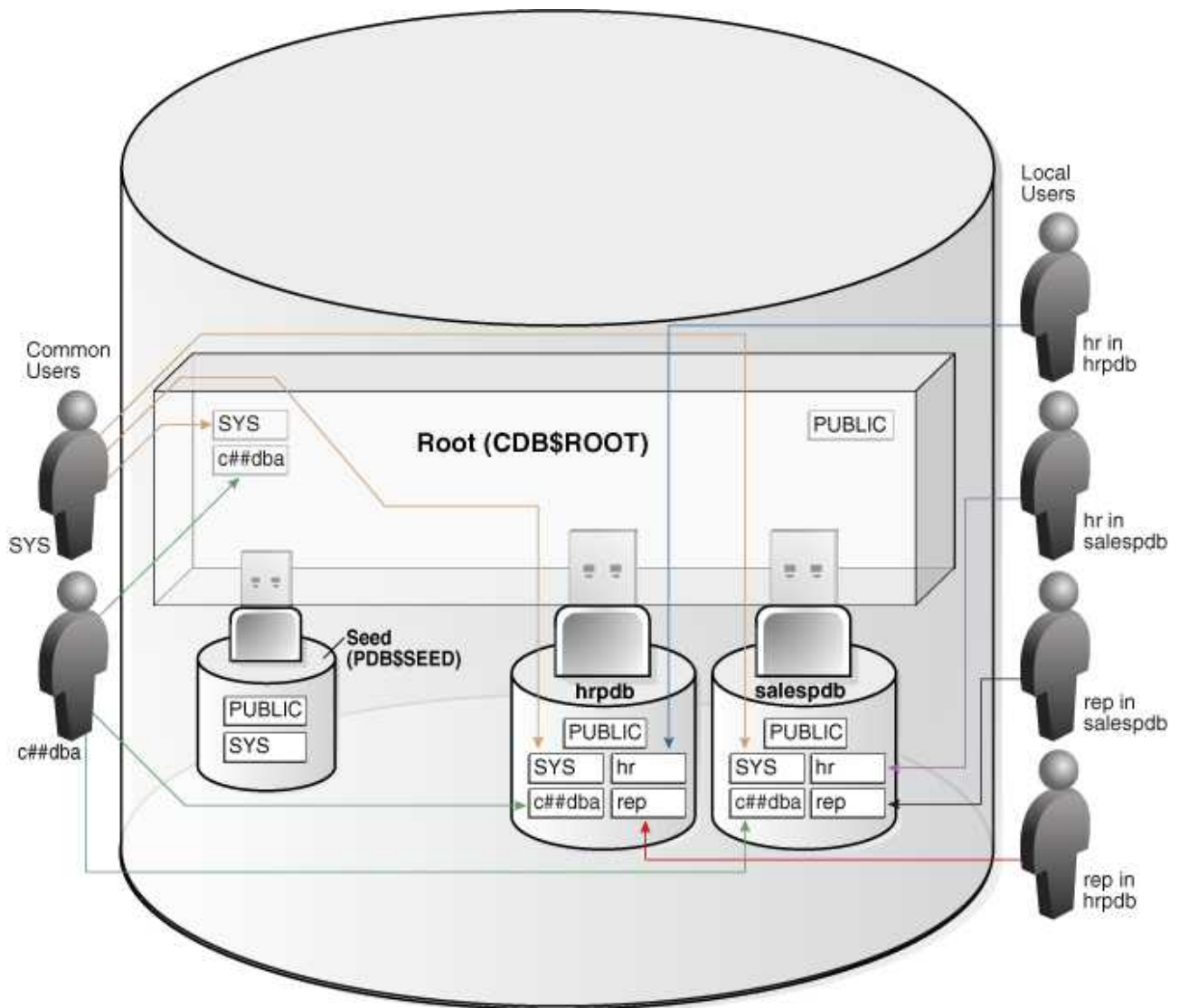
CREATE USER C##usuarioGlobal IDENTIFIED BY usuarioglobal CONTAINER=ALL;

La columna COMMON de la vista CDB_USERS muestra si es común o no. La siguiente consulta muestra el nombre del contenedor, el del usuario y si es o no común de aquellos usuarios creados con posterioridad a la creación de la base de datos:

COLUMN contenedor Format a6

COLUMN usuario Format a14

```
SELECT PDB_NAME contenedor, USERNAME usuario, COMMON FROM CDB_USERS NATURAL JOIN DBA_PDBS WHERE CREATED>(SELECT CREATED FROM V$DATABASE);
```



Un usuario común puede iniciar sesión en cualquier contenedor (incluido CDB\$ROOT) en el que tenga privilegios CREATE SESSION.

Un usuario común reside en la raíz, pero debe poder conectarse a cada PDB con la misma identidad.

Los esquemas para un usuario común pueden diferir en cada contenedor. Por ejemplo, si c##dba es un usuario común que tiene privilegios en varios contenedores, entonces el esquema en cada uno de estos contenedores puede contener objetos diferentes. Para crear un perfil en un contenedor también se usa la cláusula CONTAINER y del mismo modo que en los casos anteriores. El siguiente ejemplo crea un perfil en el contenedor XEPDB1:

```
ALTER SESSION SET CONTAINER=XEPDB1;
CREATE PROFILE PasswordEstandar LIMIT
FAILED_LOGIN_ATTEMPTS 3
PASSWORD_LIFE_TIME 189
PASSWORD_REUSE_TIME 2
PASSWORD_LOCK_TIME 1/8
PASSWORD_GRACE_TIME 3
INACTIVE_ACCOUNT_TIME 15
CONTAINER=CURRENT;
```

Edición de una cuenta de usuario

Para modificar un usuario que ya existe se usa el comando ALTER USER. Las cláusulas son las mismas que para el comando CREATE USER. El siguiente comando modifica la contraseña del usuario Gestor y se configura para que sea cambiada en la primera autenticación.

```
ALTER USER Gestor IDENTIFIED BY rotseg PASSWORD EXPIRE;
```

Para la modificación del tablespace por defecto y asignación de cuotas se usaría el siguiente comando:

```
ALTER USER Gestor DEFAULT TABLESPACE pruebas QUOTA UNLIMITED ON pruebas QUOTA
20M ON DATOS;
```

El siguiente ejemplo bloquea la cuenta Gestor y luego la desbloquea:

```
ALTER USER Gestor ACCOUNT LOCK;
ALTER USER Gestor ACCOUNT UNLOCK;
```

Borrado de una cuenta de usuario

Para eliminar una cuenta de usuario se usa el comando DROP USER. La opción CASCADE permite eliminar primero todos los objetos creados por el usuario antes de eliminar la cuenta definitivamente.

```
DROP USER nombre [CASCADE];
```

El siguiente ejemplo elimina el usuario VendedorCaja01 y todos sus objetos:

```
DROP USER VendedorCaja01 CASCADE;
```

PRÁCTICA

ALTERACIÓN DE USUARIOS

Modifica las cuentas DirectorVentas, DirectorCompras y VendedorCaja01 para cumplir:

- Se requiere cambiar la cuenta del usuario DirectorCompras. La nueva contraseña es rotceridaveun y su cuota en USERS debe ser limitada a 500 megabytes.
- Se requiere cambiar la contraseña de la cuenta DirectorVentas y que esta sea modificada en la primera conexión. La nueva contraseña es satnevaveun.
- Se requiere desbloquear la cuenta de usuario VendedorCaja01
- Borra todas las cuentas de usuario junto con sus objetos.

Concesión de privilegios

Un privilegio de usuario es el derecho a ejecutar declaraciones SQL específicas.

Los privilegios se dividen en las siguientes categorías:

- Privilegio del sistema que es el derecho a realizar una acción específica en la base de datos o realizar una acción sobre cualquier objeto de un tipo. CREATE USER, SELECT ANY TABLE y CREATE SESSION son privilegios del sistema.
- Privilegio de objeto es el derecho a realizar una acción específica sobre un objeto, por ejemplo, consultar una tabla.

CDB_TAB_PRIVS	
GRANTEE	VARCHAR2(128)
OWNER	VARCHAR2(128)
TABLE_NAME	VARCHAR2(128)
GRANTOR	VARCHAR2(128)
PRIVILEGE	VARCHAR2(40)
GRANTABLE	VARCHAR2(3)
HIERARCHY	VARCHAR2(3)
COMMON	VARCHAR2(3)
TYPE	VARCHAR2(24)
INHERITED	VARCHAR2(3)
CON_ID	NUMBER

CDB_SYS_PRIVS	
GRANTEE	VARCHAR2(128)
PRIVILEGE	VARCHAR2(40)
ADMIN_OPTION	VARCHAR2(3)
COMMON	VARCHAR2(3)
INHERITED	VARCHAR2(3)
CON_ID	NUMBER

Si un usuario concede un privilegio localmente usando la cláusula CONTAINER=CURRENT solo será efectivo en el contenedor actual. Si se usa la cláusula CONTAINER=ALL entonces el privilegio será ejercitable en todos los contenedores.

El comando que se usa para conceder un privilegio a un usuario es **GRANT TO** usuario. La sintaxis de este comando es la siguiente:

```
GRANT privilegio | ALL PRIVILEGES  
[ON [usuario.]objeto]  
TO usuario | rol | PUBLIC  
[WITH ADMIN OPTION]  
[CONTAINER=ALL | CURRENT];
```

No hace falta conocer el nombre exacto de todos los privilegios. La consulta siguiente muestra el nombre y el código de los privilegios del sistema que se pueden conceder a cualquier usuario:

```
SELECT PRIVILEGE, NAME FROM SYSTEM_PRIVILEGE_MAP ORDER BY NAME;
```

El siguiente ejemplo concede al usuario Gestor el privilegio de hacer consultas sobre cualquier objeto:

```
GRANT SELECT TO Gestor;
```

Con la opción ON se podrá indicar a qué objeto hace referencia el privilegio.

```
GRANT select ON JeFeProduccion.produccion TO Raquel;
```

Para indicar más de un privilegio, se separan por comas. Se puede indicar más de un usuario separando con comas sus nombres. ejemplo:

```
GRANT update, select, insert ON Produccion, LotesProduccion TO Andres, Susana;
```

La opción PUBLIC asigna el privilegio a los usuarios ya creados y a los que se crearán.

GRANT INSERT, UPDATE, DELETE ON Producto TO PUBLIC;

GRANT UPDATE(name) ON producto TO PUBLIC;

La opción WITH ADMIN OPTION indica que el usuario al que se le ha asignado el privilegio puede también conceder el mismo privilegio a otros usuarios.

Para conceder un rol se hace como si fuese un privilegio. Es decir

GRANT rol TO usuario.

Cuando se concede un privilegio se puede indicar en qué contenedor tiene ámbito a través de la opción **CONTAINER**. Si un privilegio se concede en el contenedor actual, este solo tiene ámbito en él. Sin embargo, si se indica que el ámbito es en todos los contenedores se define como un privilegio común a todos los usuarios. Evidentemente, para un usuario local la concesión de privilegios es local. El siguiente ejemplo concede al usuario global c##usuarioglobal el privilegio local en el contenedor actual:

GRANT UPDATE(prCoste) ON producto TO c##usuarioglobal CONTAINER=CURRENT;

Sin embargo, al mismo usuario global se le puede asignar otro privilegio para todos los contenedores. El siguiente ejemplo concede el privilegio global al usuario c##usuarioglobal en todos los contenedores:

GRANT UPDATE(prCoste) ON Producto TO c##usuariogloba1 CONTAINER=ALL;

Para conocer los privilegios locales y globales que cada usuario tiene en los diferentes contenedores se pueden consultar las vistas **CDB_TAB_PRIVS** y **CDB_SYS_PRIVS**.

La siguiente consulta muestra los privilegios de sistema que se han concedido a usuarios y a roles:

COL Destino FOR a17

COL Privilegio FOR a40

SELECT GRANTEE Destino, PRIVILEGE Privilegio, COMMON, INHERITED FROM CDB_SYS_PRIVS;

Las columnas COMMON e INHERITED muestran si la concesión es global y si fue concedida desde otro contenedor.

PRÁCTICA

GESTIÓN DE PRIVILEGIOS

Crea el usuario jefeCompras en el contenedor XEPDB1 con la contraseña sarpmoc. Tendrá 500 megabytes de limite en su tablespace predeterminado Tienda. Ejecuta el comando CONN jeFecompras/sarpmoc@xepdb1. ¿Qué ocurre?

Concédele los roles CONNECT y RESOURCE.

Conéctate como el usuario y crea una tabla. Por qué puede crear tablas? Mira la vista cdb_sys_privs para los roles del usuario

Retirada de privilegios

El comando que se usa para retirar un privilegio a un usuario es REVOKE FROM. La sintaxis de este comando es la que sigue:

REVOKE privilegio | ALL PRIVILEGES

[ON [usuario].objeto]

FROM usuario | rol | PUBLIC;

Las cláusulas son iguales a las de GRANT, pero esta vez se usa FROM en lugar de TO.

Es importante recalcar que no se pueden revocar premisos que no hayan sido dados con grant. El revoke es una especie de borrado del grant dado anteriormente.

Para consultar la información sobre los privilegios que un usuario tiene asignado se pueden usar las vistas siguientes:

DBA_SYS_PRIVS: todos los privilegios asignados a usuarios o a roles

DBA_TAB_PRIVS : privilegios concedidos sobre tablas

DBA_COL_PRIVS : privilegios concedidos sobre columnas de tablas

DBA_ROLE_PRIVS : roles concedidos a usuarios u otros roles

También tenemos que tener en cuenta que si estamos en el contenedor ROOT podemos acceder a las vistas con CDB en lugar de DBA y veríamos lo mismo para todos los contenedores.

La siguiente consulta muestra información de los privilegios de los usuarios:

```
SELECT GRANTEE, PRIVILEGE FROM DBA_SYS_PRIVS;
```

PRÁCTICA

GESTIÓN DE PRIVILEGIOS II

Con el usuario jefecompras creado anteriormente:

- Revoca el privilegio de CREATE TABLE
- Comprueba si puede crear tablas y explica por qué
- Mira todos los privilegios, roles y privilegios sobre tablas concedidos a jefecompras.
- Haz una lista de todos los privilegios concedidos a jefecompras sea directamente o a través de roles. Concédete el privilegio de create any table y repite la query.
- Revoca el rol RESOURCE
- Concede el privilegio de seleccionar de la tabla regions de HR
- Verifica en las vistas del catálogo que tiene ese privilegio concedido.
- Concede el privilegio de update en la columna region_id
- Verifica en las vistas del catálogo que tiene ese privilegio concedido.

Consulta la información de usuarios

Para consultar información de los usuarios en el diccionario de datos se pueden usar las vistas **DBA_USERS** y **DBA_TS_QUOTAS**.

La vista DBA_USERS contiene información sobre los usuarios creados en el sistema. Su descripción se muestra a continuación. Sus columnas más importantes son las siguientes:

DBA_USERS			
USERNAME	VARCHAR2(128)	EXTERNAL_NAME	VARCHAR2(4000)
USER_ID	NUMBER	PASSWORD_VERSIONS	VARCHAR2(17)
PASSWORD	VARCHAR2(4000)	EDITIONS_ENABLED	VARCHAR2(1)
ACCOUNT_STATUS	VARCHAR2(32)	AUTHENTICATION_TYPE	VARCHAR2(8)
LOCK_DATE	DATE	PROXY_ONLY_CONNECT	VARCHAR2(1)
EXPIRY_DATE	DATE	COMMON	VARCHAR2(3)
DEFAULT_TABLESPACE	VARCHAR2(30)	LAST_LOGIN	TIMESTAMP(9) WITH TIME ZONE
TEMPORARY_TABLESPACE	VARCHAR2(30)	ORACLE_MAINTAINED	VARCHAR2(1)
LOCAL_TEMP_TABLESPACE	VARCHAR2(30)	INHERITED	VARCHAR2(3)
CREATED	DATE	DEFAULT_COLLATION	VARCHAR2(100)
PROFILE	VARCHAR2(128)	IMPLICIT	VARCHAR2(3)
INITIAL_RSRC_CONSUMER_GROUP	VAR(128)	ALL_SHARD	VARCHAR2(3)

La siguiente consulta muestra información de interés sobre las cuentas de usuarios:

```
SELECT USERNAME, PASSWORD, ACCOUNT_STATUS, CREATED,
AUTHENTICATION_TYPE FROM DBA_USERS;
```

La vista DBA_TS_QUOTAS contiene información de las cuotas asignadas a los usuarios en los tablespaces.

DBA_TS_QUOTAS	
TABLESPACE_NAME	VARCHAR2(30)
USERNAME	VARCHAR2(128)
BYTES	NUMBER
MAX_BYTES	NUMBER
BLOCKS	NUMBER
MAX_BLOCKS	NUMBER
DROPPED	VARCHAR2(3)

Ejemplos de consultas:

```
SELECT TABLESPACE_NAME, USERNAME, BYTES/1024/1024 as MB
FROM DBA_TS_QUOTAS;
```

PRÁCTICA

GESTIÓN DE PRIVILEGIOS III

Indica la secuencia de comandos para realizar las siguientes tareas con relación a la gestión de cuentas de usuarios sobre objetos:

a) Verifica si está creada una cuenta llamada AyudaGestor. Si no lo está créala. Qué consulta has hecho para conocer si este usuario ya está creado?

b) Usando la cuenta SYS crea el siguiente tablespace:

```
CREATE TABLESPACE ProductosAux
```

```
DATAFILE 'c:\oradata\prodaux01.dbf' SIZE 10M REUSE, 'c:\oradata\prodaux02.dbf' SIZE 5M REUSE;
```

c) Crea la siguiente tabla:

```
CREATE TABLE ProductoAux(
    prod_ID NUMBER(3) PRIMARY KEY,
    nombre VARCHAR2(29)) TABLESPACE ProductosAux;
```

d) Asigna a dicho usuario una cuota de 1 megabyte en ese tablespace y como por defecto.

e) El usuario AyudaGestor no puede eliminar ni insertar registros en ProductoAux, pero si actualizar y consultar la tabla.

f) Permite que todos los usuarios puedan consultar la tabla ProductoAux.

Gestión de roles y perfiles

Los roles son un conjunto de privilegios que se pueden otorgar a un usuario o a otro rol. De esta forma se simplifican los procesos de concesión de privilegios.

Creación, asignación y revocación de roles

Un rol es un conjunto de privilegios recogidos con un nombre. Cuando se asigna un rol a un usuario, directamente se le conceden todos los privilegios asociados a ese rol. Lo primero que se debe hacer es crear el rol mediante el comando **CREATE ROLE**. Una vez creado, se asignan los privilegios oportunos utilizando el comando **GRANT**. Para revocar un privilegio a un rol se usa el comando **REVOKE**.

La sintaxis del comando **CREATE ROLE** es la siguiente:

CREATE ROLE nombre_rol;

Opcionalmente, a un rol se le puede asignar una clave, pero entonces no será un rol por defecto y el usuario tendrá que establecerlo

CREATE ROLE nombre_rol IDENTIFIED BY contraseña;

SET ROLE nombre_rol IDENTIFIED BY contraseña;

Del mismo modo que se asigna privilegios a un usuario, para un rol se usa el comando **GRANT** con la sintaxis siguiente:

GRANT nombre_rol TO usuario;

Default roles

Cuando creamos un rol, excepto si es con contraseña, Oracle lo establece como rol por defecto, es decir, puede ser usado y estará activado por defecto. La columna `default_role` de la vista `dba_role_privs` nos dirá si es por defecto o no.

Si alteramos el usuario y le ponemos uno o varios roles por defecto se los activamos para que sean usados al iniciar sesión, los otros que tenga ya no lo serán

ALTER USER usuario DEFAULT ROLE rol1, rol2 | ALL;

SELECT * FROM SESSION_ROLES; -- nos dice los que están activos

Un rol que no esté activado por defecto, para ser usado tiene que el usuario activarlo con **SET ROLE rol;**

SET ROLE DEFAULT; Restaura los roles predeterminados

Creación de roles comunes

Se pueden crear roles comunes, siempre que se cumplan las siguientes condiciones.

- Debes estar conectado como usuario común con los privilegios CREATE ROLE y SET CONTAINER otorgados comúnmente.
- El contenedor actual debe ser el contenedor raíz.
- El nombre del rol común debe tener el prefijo "C##" o "c##" y contener solo caracteres ASCII o EBCDIC.
- El nombre del rol debe ser único en todos los contenedores.
- El rol se crea con la cláusula CONTAINER=ALL

Asignación de roles comunes

La diferencia entre asignar roles global o localmente es usando la cláusula **CONTAINER**, el usuario que concede, el concedido y el rol deben ser comunes:

GRANT CREATE SESSION TO c##test_user1 CONTAINER=ALL; --comun

GRANT CREATE SESSION TO test_user3; -- local

usuarios locales no deben tener roles ni privilegios comunes y roles y privilegios locales no deben ser concedidos de manera global.

Sinónimos

Un sinónimo en Oracle, es una representación local o pública de un objeto perteneciente a un esquema. Sirve para poder hacer referencia a aquel objeto sin tener que anteponer su esquema. Un sinónimo público puede ser visto por todos los usuarios, pero uno privado, sólo por el usuario que lo creó. Por ejemplo permiten referirnos a la tabla regions directamente poniendo el sinónimo (como un alias) sin poner hr.regions

CREATE [PUBLIC] SYNONYM <nome> FOR <objeto>

PRÁCTICA

SINONIMOS

Crea una vista en el usuario hr con solo el nombre de las regiones. Crea un sinónimo público y uno privado con otro usuario. Da permisos y mira como se accede a las tablas directamente y usando el sinónimo

PRÁCTICA

DEFAULT ROLES

Queremos un usuario llamado juan y tres roles: analista (puede consultar datos de hr.employees), gestor (también puede modificar datos de esa tabla), comercial (solo puede seleccionar e insertar de/en hr.departments) y auditor (puede incluso borrar datos en ambas tablas).

Queremos que analista y comercial sean roles activados por defecto mientras que gestor no lo sea y deba activarse manualmente. Auditor debe ser creado con contraseña.

Crea el escenario y prueba a insertar un departamento, borrarlo, cambiar el sueldo de un empleado activando y desactivando roles.

PROXY USER

Hay dos formas de que un usuario "actúe como otro": Podemos crear un rol que contenga los privilegios que queremos que tenga y asignamos el rol a ambos usuarios. Podemos querer actuar como él (conectarnos como si fuese él) Usaremos la cláusula proxy

ALTER USER origen GRANT CONNECT THROUGH usuario;

Así usuario podrá conectarse como origen

CONNECT usuario[origen]/password_usuario; -- usuario establecerá sesión como origen

PRÁCTICA

GRANTS COMUNES Y LOCALES

Explica qué hace cada una de estas sentencias y si es local o común y los resultados que se obtienen

```
#> CONNECT SYSTEM/oracle@XE
Connected.
```

```
SQL> CREATE USER c##dba IDENTIFIED BY password
CONTAINER=ALL;
SQL> GRANT CREATE SESSION TO c##dba;
SQL> CREATE ROLE c##admin CONTAINER=ALL;
SQL> GRANT SELECT ANY TABLE TO c##admin;
SQL> GRANT c##admin TO c##dba;
SQL> EXIT;
SQL> CONNECT c##dba/oracle@hrpdb
ERROR: ORA-01045: user c##dba lacks CREATE SESSION privilege;
logon denied
```

```
SQL> CONNECT SYSTEM/oracle@hrpdb
Connected.
SQL> GRANT CONNECT, RESOURCE TO c##dba;
Grant succeeded.
SQL> EXIT
```

```
SQL> CONNECT c##dba/oracle@hrpdb
Connected.
SQL> SELECT COUNT(*)
FROM hr.employees;
select * from hr.employees
ERROR at line 1: ORA-00942: table or view does not exist
```

```
SQL> CONNECT SYSTEM/oracle@root
Connected.
SQL> GRANT SELECT ANY TABLE TO c##admin CONTAINER=ALL;
Grant succeeded.
SQL> CONNECT c##dba/oracle@hrpdb
Connected.
SQL> SELECT COUNT(*) FROM hr.employees;
select * from hr.employees
ERROR at line 1: ORA-00942: table or view does not exist
```

```
SQL> CONNECT SYSTEM/oracle@root
Connected.
SQL> GRANT c##admin TO c##dba CONTAINER=ALL;
Grant succeeded.
SQL> CONNECT c##dba/oracle@hrpdb
Connected.
SQL> SELECT COUNT(*)
FROM hr.employees;
```

```
COUNT(*)
-----
```

PRÁCTICA

Práctica final

Estás a cargo de la administración de una base de datos en Oracle para un hospital ficticio llamado Cunqueiro. El hospital necesita implementar un sistema de usuarios y roles para gestionar el acceso a los datos. A continuación, se describen los tipos de usuarios

- Hay tres tipos de usuarios: medicos, enfermeros y administrativos

En cuanto a las lecturas:

los médicos pueden : acceder a todos los datos

los enfermeros sólo pueden acceder a todos los datos de los pacientes, ingresos y los tratamientos_realizados excepto el campo observaciones al que solo pueden acceder los médicos

los administrativos pueden: acceder a los datos de los pacientes e ingresos.

En cuanto a las inserciones y actualizaciones:

los médicos pueden insertar y actualizar cualquier campo en/de cualquier tabla salvo de la tabla tratamientos

los enfermeros pueden insertar nuevos pacientes, ingresos y tratamientos_realizados así como modificarlos (excepto el campo observaciones, claro).

los administrativos solo pueden insertar nuevos pacientes e ingresos pero no pueden modificar los datos, necesitan que alguien con más permisos lo haga.

2.1 Crea un usuario global en cuyo nombre aparezca la palabra cunqueiro. Con este usuario crea el entorno de tablas que está en el fichero hospital.sql. Estas tablas deben crearse en el tablespace cunqueiro que ya está creado.

2.2 Crea el entorno de permisos adecuado a las condiciones anteriores teniendo en cuenta que ninguno de los usuarios debe poder crear, borrar, actualizar objetos (tablas)

2.3 Crea los siguientes usuarios:

pepis (medico)

mariola (enfermera)

sabina (administrativa)

asegúrate de que pepis puede ejecutar EXACTAMENTE las siguientes sentencias

```
select * from tratamientos_realizados;
```

```
update tratamientos_realizados set observaciones='Dar de alta' where id_tratamiento_realizado=7;
```

asegúrate de que mariola puede ejecutar EXACTAMENTE las siguientes sentencias

```
select * from tratamientos_realizados; -- no debe aparecer el campo observaciones
```

```
update ingresos set fecha_alta=sysdate where id_ingreso=7;
```

asegúrate de que sabina puede ejecutar EXACTAMENTE las siguientes sentencias

```
select * from pacientes join ingresos using(id_paciente);
```

3. Concede a Mariola el permiso de update del campo observaciones. Haz una sentencia en la que muestres todos los permisos sobre las tablas concedidos al usuario mariola tanto directamente como a través de roles.